Лекция 7. Шаблоны

1. Шаблоны в C++
2. Шаблоны функций
   - Явное указание типа шаблона
3. Шаблоны классов
   - Определение метода вне класса
   - CTAD
4. Шаблоны псевдонимов (C++11)
   - Использование псевдонимов
5. Шаблоны переменных (C++14)
6. Параметры по умолчанию для шаблона
7. Специализация шаблона
   - Частичная специализация шаблона
   - Удалить специализацию шаблона
8. Явное инстанцирование
9. Зависимые имена
10. Шаблон в параметрах шаблона
11. Non-type параметры шаблона
    - Использование `auto` в параметрах шаблона
12. Вывод типов в шаблонах
    - Передача по значению `T`
    - Передача по значению `const T`
    - Передача по ссылке `T&`
    - Передача по ссылке `const T&`
    - Передача по указателю `T*`
    - Передача по указателю `const T*`
    - Правила свертки ссылок (схлопывания ссылок)
    - Передача по rvalue ссылке `T&&`. Forwarding references
    - Идеальная передача (prefect forwarding)
13. Вывод типов с использованием `auto`
14. Вывод типов с помощью `decltype`
15. Variadic Template (C++11)
    - Рекурсивный вызов
    - Паттерны свертки (Fold expression) C++17
    - Прозрачная функция

Шаблоны в C++

Язык C++ имеет статическую типизацию, поэтому на этапе компиляции должны быть известны типы сущностей языка. Безусловно, бывают ситуации, когда нужна функция, которая выглядит идентично за исключением только типов переменных.

```
void Swap(int& lhs, int& rhs) {
    int temp = lhs;
    lhs = rhs;
    rhs = temp;
}
```

Если нужно написать функцию для типа `double`, то приходится перегружать функцию, копируя код, лишь изменяя типы.

```
void Swap(double& lhs, double& rhs) {
    double temp = lhs;
    lhs = rhs;
    rhs = temp;
}
```

Такая функция может потребоваться и для других типов, что конечно несколько удручает

```
void Swap(int& lhs, int& rhs);
void Swap(float& lhs, float& rhs);
void Swap(double& lhs, double& rhs);
void Swap(std::string& lhs, std::string& rhs);
```

В данном случае нарушается один из популярных принципов программирования - **DRY** (Don't Repeat Yourself)

Специально для таких ситуаций язык C++ имеет шаблоны. Шаблон представляет собой обобщенное описание кода какой-либо сущности C++.

Шаблон можно определить для следующих сущностей:

- функция
- класс
- псевдоним типа (C++11)
- переменная времени компиляции (C++14)

Синтаксис `template <typename T, typename Iter>`

- `template` - ключевое слово
- `<>` - внутри треугольных скобок, указываются параметры шаблона через запятую `,`
- `typename` - ключевое слово, после которого вводится имя обобщенного типа (также можно использовать эквивалентное слово `class`, но поскольку параметром может быть и тривиальный тип, некоторые разработчики предпочитают `typename`).
- `T`, `Iter` - произвольные имена обобщенных типов, которые можно использовать в определении соответствующей сущности.

Поскольку в языке C++ имеет статическую типизацию, то на этапе компиляции необходимо знать все используемые типы. Встречая в коде использование шаблона, компилятор генерирует на основе шаблона реализацию функцию или класса для конкретного типа, с которым используется шаблон. Такой процесс называется инстанцированием шаблона (*template instantiation*).

Инстанцирование шаблона - это процесс генерации компилятором реального кода функции или класса на основе описанного ранее шаблона путем подстановки в него конкретных типов данных или значений.

Шаблоны функций

Напишем для примера обобщенную (шаблонную) функцию `Swap`

```
template <typename T>
void Swap(T& lhs, T& rhs) {
    T temp = std::move(lhs);
    lhs = std::move(rhs);
    rhs = std::move(temp);
}
```

- не забудем про `std::move`, что сделает функцию более производительной для составных типов, поддерживающих семантику перемещения

Использование шаблонной функции может ничем **НЕ** отличаться от обычной функции

```
int x = 5, y = 10;
Swap(x, y); // compiler deduces T = int

std::string x = "one", y = "two";
Swap(x, y); // compiler deduces T = std::string
```

Компилятор сгенерирует две отдельные функции (инстанцирует шаблон для типа `int` и `std::string`, создаст экземпляр шаблона функции):

```
void Swap<int>(int& lhs, int& rhs) {
    int temp = std::move(lhs);
    lhs = std::move(rhs);
    rhs = std::move(temp);
}

void Swap<std::string>(std::string& lhs, std::string& rhs) {
    std::string temp = std::move(lhs);
    lhs = std::move(rhs);
    rhs = std::move(temp);
}
```

- если шаблон есть, а его использования нет, то никаких функций сгенерировано **НЕ** будет и код шаблона не попадет в объектный файл

Явное указание типа шаблона

При выводе типов должно быть точное совпадение с шаблоном. Рассмотрим следующий пример:

```
template <typename T>
T Min(T lhs, T rhs) {
    return (lhs < rhs) ? lhs : rhs;
}
```

В таком случае, компилятор не сможет вывести тип шаблона, если аргументы разные:

```
int x = 5, y = 10;
auto result = Min(x, y); // ok T = int

double x = 2.5, y = 4.0;
auto result = Min(x, y); // ok T = double

int x = 3;
double y = 5.0;
auto result = Min(x, y); // compilation error T = (int or double) ?
```

Но если один тип можно привести к другому, то можно явно указать параметр шаблона при вызове, тем самым подсказать компилятору какой тип необходимо использовать:

```
int x = 3;
double y = 5.5;
auto result_i = Min<int>(x, y); // ok T = int
auto result_d = Min<double>(x, y); // ok T = double
```

- в первом случае `double` будет приведен к `int`, а во втором наоборот

Сейчас параметр в функцию передается по значению, что может оказаться нежелательным, поскольку вызовет копирование в случае тяжелого класса. К шаблону типа можно также добавлять cv-валификаторы и ссылки (уточнять тип). Поэтому стоит продумывать заранее с какими типами может использоваться шаблон:

```
template <typename T>
const T& Min(const T& lhs, const T& rhs) {
    return (lhs < rhs) ? lhs : rhs;
}
```

- теперь строки не будут копироваться

Следует отметить, что тело шаблона накладывает ограничения на используемые типы. Так в нашем случае для типа обязательно должен быть определен оператор `<`, чтобы возможно было инстанцировать шаблон, а если такого оператора нет, то будет получена ошибка компиляции.

В C++20 появились концепты, которые позволяют явно прописывать требования к типам.

Шаблоны классов

Шаблоны можно применять не только к функциям, но и к классам, на чем и построены контейнеры стандартной библиотеки **STL**. Синтаксис объявления шаблонного класса аналогичен.

```
template <typename T1, typename T2>
struct Pair {
    void Swap(Pair& other) noexcept;
    // ... other methods
    T1 first;
    T2 second;
};
```

Обратим внимание, что для типа `Pair` в аргументах метода `Swap` не нужно писать `Pair<T1, T2>`, потому что работает механизм *injected-class-name* (внедренное имя класса). Поскольку мы находимся внутри области видимости класса, то компилятор автоматически подставит параметры шаблоны при инстанцировании класса. Это так называемый синтаксический сахар, который позволяет избежать ошибок в написании типа.

В отличие от шаблона функции компилятор **НЕ** может вывести тип автоматически для шаблона класса, поэтому необходимо явно указывать тип в угловых скобках `<>`

```
Pair<std::string, int> p;
```

Определение метода вне класса

Для определения методов вне класса необходимо повторить шаблон и указать `Pair<T1, T2>`, чтобы показать в какой области видимости метод `Swap`, в самих параметрах функции уже можно не указывать, так как находясь внутри области видимости класса снова действует *injected-class-name*.

```
template <typename T1, typename T2>
void Pair<T1, T2>::Swap(Pair& other) noexcept {
    // some code
}
```

Если функция шаблонная, то придется дважды указывать `template`:

```
template <typename T> class SomeClass {
    const T& source_;
public:
    template <typename R>
    R Method();
};

template <typename T>
template <typename R>
R SomeClass<T>::Method() {
    R result = source_.template Method<R>();
    return result;
}
```

CTAD

На самом деле в C++17 появился CTAD (Class Template Argument Deduction) - вывод типа для шаблонов класса. Но он работает с ограничениями.

Если у класса есть конструктор, по которому можно вывести параметр шаблона, то компилятор выводит тип из аргументов конструктора при создании объекта.

Компилятор создает *deduction guides* (правила вывода типа) для конструкторов вроде такого `Box(T) -> Box<T>;`

Но можно написать и свои

```
Box(const char*) -> Box<std::string>;
Box(int) -> Box<int>;
```

Шаблоны псевдонимов

В C++11 появляется шаблоны для псевдонимов `using`.

В этом большое преимущество `using` перед `typedef`, который **НЕ** может быть шаблонным.

Можно получить псевдоним для шаблонной функции или класса:

```
template <typename T>
using Func = T(*)(T, T);

template <typename T>
using Vec = std::vector<T>;

template <typename T>
using NameToValue = std::map<std::string, T>;
```

```
template <typename T>
using Ptr = std::unique_ptr<T>;
```

Использование псевдонимов

Синтаксис псевдонимов:

```
typedef int Currency;
using Currency = int;
```

Вспомним зачем нужны псевдонимы. Наиболее часто можно встретить использование псевдонимов там, где хочется получить короткое понятное (со смысловой нагрузкой) имя, вместо длинного типа контейнера, умного указателя или итератора:

```
using t_sp_controller = std::shared_ptr<ScmController>;
using t_cit_names = std::vector<std::string>::const_iterator;
using t_name_to_info = std::unordered_map<std::string, std::string>;
```

- имена бывают ещё более длинными

Конечно, не стоит этим злоупотреблять, так как сразу увидев тип, используемый контейнер, значительно проще вспомнить какие можно вызывать методы для работы с переменной данного типа.

Также, псевдонимы удобно использовать, когда есть вероятность того, что в будущем тип может поменяться, например, сейчас данные хранятся в виде вектора байт `std::vector<uint8_t>`, но пока это только набросок решения и в будущем может быть использован либо другой контейнер (`std::deque`), либо другой тип (`char`).

```
using t_raw_data = std::vector<uint8_t>;
```

- можно и `raw_data_t`, всё зависит от стайлгайда

Использование псевдонима позволит отличить вектор байт в какой-то функции от вектора байт, который скрывает сырые данные в определенном порядке. Что позволит легко изменить тип конкретно для наших сырых данных в одном месте и не думать лишний раз относится ли появление в какой-то функции `std::vector<uint8_t>` к сырым данным.

Кроме того, псевдонимы используются для функций:

```
using FuncCalc = double(*)(double, double);
typedef double(*FuncCalc)(double, double);
```

- в данном случае `using` уже приятнее

В стандартной библиотеке с C++14 применяется для типов:

```
template <typename T>
using remove_reference_t = typename remove_reference<T>::type;
```

Шаблоны переменных (C++14)

В C++14 появилась возможность создавать шаблоны переменных времени компиляции.

```
template <typename T>
constexpr T max_value = std::numeric_limits<T>::max();

template<typename T>
constexpr T pi = static_cast<T>(3.1415926535897932385L);
```

Как правило, переменные помечены ключевым словом `constexpr`.

Благодаря шаблонам, можно написать программу, которая на этапе компиляции программирует программу на поведение в зависимости от принимаемого типа.
Это называется метапрограммирование.

Например, можно определить является ли тип указателем.
До C++14 для аналогичных целей использовали структуру со статическим полем.

```
template <typename T>
struct is_pointer {
    static constexpr bool value = false;
};

template <typename T>
struct is_pointer<T*> {
    static constexpr bool value = true;
};
```

- базовый шаблон помечается `false`, а частичная специализация, отвечающая требованиям `true`

Использование:

```
is_pointer<int*>::value // true
is_pointer<int>::value  // false
```

Именно так реализована шаблоны из заголовочного файла `<type_traits>`

Начиная с C++14 можно создать непосредственно шаблоны переменной:

```
template <typename T>
constexpr bool is_pointer_v = false;

template <typename T>
constexpr bool is_pointer_v<T*> = true;
```

Использование несколько упростилось:

```
is_pointer_v<int*>   // true
is_pointer_v<int>    // false
```

В C++17 в стандартную библиотеку добавлены хэлперы в виде шаблонных переменных. Поскольку Legacy код тоже должен работать, это сделано через соответствующие шаблонные структуры:

```
template <typename T>
inline constexpr bool is_integral_v = is_integral<T>::value
```

В современном C++ принято использовать варианты оканчивающиеся на `_v`, `_t`.

Примеры из стандартной библиотеке:

```
std::is_integral_v<int>;     // true
std::is_same_v<int, double>; // false
```

Активно используется в SFINAE - Substitution Failure Is Not An Error
(Провал подстановки типа в шаблон это не ошибка)

Параметры по умолчанию для шаблона

Можно указывать параметры шаблона по умолчанию через оператор `=`.

Аналогично параметрам по умолчанию для аргументов функции, параметры по умолчанию для шаблона можно указывать справа налево.

```
template <typename T, typename Allocator = std::allocator<T>>
class Vector {};
```

Эта возможность широко применена в стандартной библиотеке:

- В контейнерах есть параметр по умолчанию для аллокатора.
- У умных указателей есть стандартный удалитель.
- У адапторов есть параметр по умолчанию для используемого контейнера.

Начиная с C++11 параметр по умолчанию возможно задать для шаблона функции

```
template <typename T = int>
T Make() { return T{}; }
```

Специализация шаблона

Может возникнуть ситуация, когда возникает необходимость изменить тело функции для определенного типа, например, для строки в реализации хочется сравнивать только размер или сравнивать строки другим способом (**НЕ** лексикографически). Шаблон можно специализировать для определенного типа.

Специализация шаблона - версия шаблона для определенного типа.

Следовательно, можно дать более короткое определение для инстанцирования. Инстанцирование - процесс порождения экземпляра специализации шаблона.

Специализация шаблона для типа `std::string`

```
template <>
const std::string& Min<std::string>(const std::string& lhs, const std::string&
rhs) {
    return (lhs.size() < rhs.size()) ? lhs : rhs;
}
```

Может возникнуть желание написать обычную перегрузку функции и так тоже можно:

```
const std::string& Min(const std::string& lhs, const std::string& rhs);
```

Теперь компилятор будет вызывать данную перегрузку при использовании `Min` для строки. Но это уже **НЕ** шаблон, это перегрузка. И, если другая шаблонная функция, которая использует шаблон `Min<T>` внутри себя, то для строки перегрузка не сможет быть вызвана.

Поиск подходящей функции происходит в соответствии с порядком поиска по имени:

- Обычные функции (точное совпадение лучше, создания экземпляра)
- Специализация шаблонной функции
- Шаблонная функция

Перегрузка шаблонной функции может также быть шаблоном, например с большим количеством
параметров:

```
template <typename T>
const T& Min(const T& a, const T& b, const T& c) {
    return Min(Min(a, b), c);
}
```

Частичная специализация шаблона

Частичная специализация **НЕ** разрешена для функций, но разрешена для классов

- Должна быть действительно менее общей, чем шаблон общего вида
- Следовать после шаблона общего вида*

```
template <typename T, typename U> class SomeClass {};
// частичные специализации
template <typename T> class SomeClass<T, T> {};
template <typename T> class SomeClass<T, double> {};
template <typename T, typename U> class SomeClass<T*, U*> {};
```

Частичная специализация может быть более специальной, и иметь больше шаблонных параметров:

```
template <typename T> class SomeClass {};
template <typename R, typename Arg> class SomeClass<R(Arg)> {}; // std::function
```

Частичная специализация для указателя:

```
template <typename T>
struct IsPointer {
    static constexpr bool value = false;
};

template <typename T>
struct IsPointer<T*> {
    static constexpr bool value = true;
};
```

- так можно на этапе компиляции получить информацию о том является ли тип указателем

Удалить специализацию шаблона

Можно запретить инстанцирование шаблона для определенного типа.

Начиная с C++11 можно использовать ключевое слово `delete`

```
template <>
class SomeClass<double> = delete;   // C++11
```

Можно было запретить определенный тип в классе с помощью `static_assert`

```
template <typename T>
class SomeClass {
```

```
    static_assert(!std::is_same<T, double>::value, "Not allowed");
};
```

- далее будет рассмотрено, что представляет `std::is_same_v<T, double>`

Явное инстанцирование

Шаблоны инстанцируются неявно при первом использовании (implicit instantiation).

Явное инстанцирование (explicit instantiation) позволяет принудительно создать (или запретить создание) шаблона в единице трансляции.

Явное инстанцирование следует правилу **ODR** - должно быть единственным в программе

Синтаксис для функций:

```
template int Min<int>(int, int); // explicit instantiation
```

Синтаксис для классов:

```
template class std::vector<int>; // explicit instantiation
```

Можно отдельно инстанцировать метод класса:

```
template void SomeClass<int>::SomeMethod();
```

Запретить явное инстанцирование в текущей единице трансляции можно, добавив ключевое слово `extern` (C++11):

```
extern template int Min<int>(int, int);
extern template class SomeClass<int>;
```

Как правило запрет явного инстанцирования помещают в *header*, а единственное явное инстанцирование помещают в *cpp* файл реализации. Что позволяет уменьшить время компиляции, когда шаблоны инстанцируются неявно в разных единицах трансляции и затем склеиваются компилятором.

ЗАПРЕТ явного инстанцирования **НЕ** является полным запретом инстанцирования данного шаблона с определенным типом. Для запрета инстанцирования используется удаление специализации шаблона

Правила инстанцирования:

- Явное инстанцирование после полного определения шаблона
- Явное инстанцирование единожды в программе
- Явная специализация единожды в программе

- Явное инстанцирование должно быть после явной специализацией

Зависимые имена

Зависимое имя - имя, которое зависит от параметров шаблона

```
struct TypeWithType {
    using value_type = int;
    static const int value = 18;
};

template <typename T>
struct Ptr {
    typename T::value_type* ptr;
};

Ptr<TypeWithType> a;
```

```
template <typename Container>
struct It {
    typename Container::const_iterator it;
};

It<std::vector<int>> it;
```

Компилятор, встречая зависимое имя от шаблона **НЕ** знает, является оно типом или именем переменной.

Нужно писать `typename` всегда, когда происходит обращение к зависимому имени, которое должно быть интерпретировано как тип.

```
template <typename Container>
void PrintElements(const Container& cont) {
    typename Container::const_iterator it = cont.begin();
    for (; it != cont.end(); ++it) {
        std::cout << *it << std::endl;
    }
}

std::vector<int> vec = {1, 2, 3};
std::list<double> lst = {1.1, 2.2, 3.3};

PrintElements(vec);  // Container = std::vector<int>
PrintElements(lst);  // Container = std::list<double>
```

Если нужно обратиться к зависимому имени, которое является шаблоном, то добавляется ещё ключевое слово `template` при обращении к шаблону

```
template <typename T>
struct Outer {
    template <typename U>
    struct Inner {
        U value;
    };
};

template <typename T>
void process() {
    typename Outer<T>::template Inner<int> obj;
    obj.value = 18;
}
```

```
struct Processor {
    template<size_t N, typename U>
    void process(U value) {
        std::cout << "Processing: " << N << " count " << std::endl;
        for (size_t count = 0; count < N; ++count) {
            std::cout << "Process: " << value << std::endl;
        }
    }
    void process(int x) {
        std::cout << "Process: " << x << std::endl;
    }
};
```

Шаблон в параметрах шаблона

В параметрах шаблона можно указать шаблон

```
template <template <typename> typename Container, typename T>
void Print(const Container<T>& cont);
```

Non-type параметры шаблона

Шаблон может принимать не только типы, но и нетипизированные параметры, которые представляют собой значения известные на этапе компиляции:

- Интегральные типы
- Указатели, указатели на члены
- Ссылки, ссылки на массивы
- Перечисления

- `std::nullptr_t` (C++11)
- `auto` (C++17)
- Числа с плавающей точкой (C++20)
- Структурные типы (C++20), у которых все поля и базовые классы *public* и **НЕ** *mutable*. При этом всем поля и поля всех базовых классов тоже структурные типы или массивы

```
template <typename T, size_t Size>
class Array {
public:
    size_t Size() const { return Size; }
    T& operator[](int index) { return data[index]; }
    // ... other methods
private:
    T data[Size]; // Size is known in compile time
};
```

Использование `auto` в параметрах шаблона

Начиная с C++17 можно использовать `auto` для вывода типа *non-type* параметра

```
template <typename T, auto Size>
class Array {
    // ...
};

int main() {
    Array<int, 10> a1;      // Size has type = int
    Array<int, 10ull> a2;    // Size has type = unsigned long long
}
```

Вывод типов в шаблонах

Чрезвычайно важно понять как действует вывод типов в шаблонах.

Передача по значению `T`

Шаблон, имеющий аргумент типа `T`, принимает аргумент по значению

```
template <typename T>
void func(T arg);
```

При этом действуют следующие правила:

- Срезаются ссылки (отбрасываются)
- Срезаются верхнеуровневые cv-квалификаторы
- Происходит array-to-pointer преобразование для массивов
- Происходит преобразование function-to-fuction-pointer для функций

```
int val = 18;
int& ref = val;
const int cval = val;
const int& cref = val;
const int&& crref = 18;

func(val);        // T -> int
func(ref);        // T -> int
func(cval);       // T -> int
func(cref);       // T -> int
func(crref);      // T -> int
func(18);         // T -> int

int* ptr = &val;
const int* ptr_c = &val;
const int* const cptr_c = &val;
int arr[5];
void f();

func(ptr);        // T -> int*
func(ptr_c);      // T -> const int*
func(cptr_c);     // T -> const int*
func(arr);        // T -> int*
func(f);          // T -> void(*)()
```

Передача по значению `const T`

Шаблон, имеющий аргумент типа `const T`, принимает аргумент по значению с квалификатором `const`

```
template <typename T>
void func(const T arg);
```

Правила вывода типов аналогичны `T`, но после вывода добавляется верхнеуровневый `const`. При передаче указателя получиться, что константным станет указатель!

Главная идея в том, то `const` применяется к выведенному типу `T` и если вывелся указатель, то константным становится указатель

```
int& ref = val;

func(ref);        // T -> int, const T -> const int
```

```
int* ptr = &val;
const int* ptr_c = &val;
const int* const cptr_c = &val;
int arr[5];
void f();

func(ptr);        // T -> int*, const T -> int* const
func(ptr_c);      // T -> const int*, const T -> const int* const
func(cptr_c);     // T -> const int*, const T -> const int* const
func(arr);        // T -> int*, const T -> int* const
func(f);          // T -> void(*)(), const T -> void(* const)()
```

Следует отметить, что такой шаблон как правило бесполезен

Передача по ссылке `T&`

Шаблон принимает аргумент по ссылке

```
template <typename T>
void func(T& arg);
```

При этом действуют следующие правила:

- Параметр `T` выводиться без ссылки, но `T&` имеет ссылку
- CV-квалификаторы не отбрасываются
- Массивы **НЕ** преобразуются к указателю
- Функции **НЕ** преобразуются к указателю на функцию

```
int val = 18;
int& ref = val;
const int cval = val;
const int& cref = val;
const int&& crref = 18;

func(val);        // T -> int, T& -> int&
func(ref);        // T -> int, T& -> int&
func(cval);       // T -> const int, T& -> const int&
func(cref);       // T -> const int, T& -> const int&
func(crref);      // T -> const int, T& -> const int&
func(18);         // compilation error

int* ptr = &val;
const int* ptr_c = &val;
const int* const cptr_c = &val;
int arr[5];
void f();

func(ptr);        // T -> int*, T& -> int*&
func(ptr_c);      // T -> const int*, T& -> const int*&
```

```
func(cptr_c);      // T -> const int*, T& -> const int* const&
func(arr);         // T -> int[5], T& -> int(&)[5]
func(f);           // T -> void(), T& -> void(&)()
```

Передача по ссылке `const T&`

Шаблон принимает аргумент по ссылке с квалификатором `const`

```
template <typename T>
void func(const T& arg);
```

Важно понимать, что `const` применяется также к выведенную типу.

Рассмотрим пример с указателем.

```
template <typename T>
void func(const T& ptr)
{
    *ptr = 10;
}

int val = 18;
int* ptr = &val;
func(ptr);
```

- Такой код скомпилируется и значение `val` изменится, так как `T` выведется в тип `int*`, и примениться константность к указателю `int* const`

```
template <typename T>
void func(const T& ptr)
{
    ptr = nullptr;
}

int val = 18;
int* ptr = &val;
func(ptr);
```

- Такой код **НЕ** скомпилируется, ошибка будет говорить о том, что нельзя присваивать значение *read-only* объекту

Подытожим по примеру:

```
int val = 18;
int& ref = val;
const int cval = val;
const int& cref = val;
const int&& crref = 18;
```

```
func(val);        // T -> int, const T& -> const int&
func(ref);        // T -> int, const T& -> const int&
func(cval);       // T -> const int, const T& -> const int&
func(cref);       // T -> const int, const T& -> const int&
func(crref);      // T -> const int, const T& -> const int&
func(18);         // T -> int, const T& -> const int&

int* ptr = &val;
const int* ptr_c = &val;
const int* const cptr_c = &val;
int arr[5];
void f();

func(ptr);        // T -> int*, const T& -> int* const&
func(ptr_c);      // T -> const int*, const T& -> const int* const&
func(cptr_c);     // T -> const int*, const T& -> const int* const&
func(arr);        // T -> int[5], const T& -> const int(&)[5]
func(f);          // T -> void(), const T& -> void(&)()
```

Передача по указателю `T*`

Шаблон принимает аргумент по указателю. Аргумент не имеет ссылки, поэтому передается копия указателя. Таким образом внешний указатель нельзя изменить

```
template <typename T>
void func(T* arg);
```

Принимается адрес для переменных или непосредственно указатель.

При этом действуют следующие правила:

- Параметр `T` выводиться без указателя (снимается один уровень косвенности), но `T*` является указателем. При передаче указателя на указатель `T = T*`, тогда `T* = T**`
- CV-квалификаторы не отбрасываются
- Массивы естественно принимаются по указателю

Передача по указателю `const T*`

Шаблон принимает аргумент по указателю. Аргумент не имеет ссылки, поэтому передается копия указателя. Таким образом внешний указатель нельзя изменить

```
template <typename T>
void func(const T* arg);
```

Будет ошибка при передаче функции:

```
void f();
func(f);          // compilation error
// template argument deduction/substitution failed:
// types ‘const T’ and ‘void()’ have incompatible cv-qualifiers
```

Правила свертки ссылок (схлопывания ссылок)

В обычных функциях, если в качестве типа принимается `&&`, то вызов разрешен от временных объектов, а если `&`, то от *lvalue*.

Но при написании шаблонной функции хочется, чтобы обобщенная функция работала бы как с *rvalue* объектом, так и с *lvalue* объектом, в зависимости от того, что передается в функцию. И когда функция имеет несколько параметров не было бы необходимости писать множество перегрузок для комбинаций `&&` и `&` у разных параметров.

В контексте вывода типов ссылки сворачиваются (reference collapsing) и левая ссылка выигрывает, если она есть

Inner | Outer | Result
T& | T& | T&
T& | T&& | T&
T&& | T& | T&
T&& | T&& | T&&

Передача по rvalue ссылке `T&&`. Forwarding references

Когда появляется контекст вывода типов, то rvalue ссылка `&&` имеет особое поведение. Такую ссылку стали называть универсальной ссылкой или *forwarding reference*.

```
template <typename T>
void func(T&& x);
```

- есть контекст вывода типа

НЕ путать с обычной функцией принимающей rvalue ссылку:

```
void func(int&& x);
```

- нет контекста вывода и соответствующих правил

Наличие `T&&` в контексте вывода заставляет компилятор выводить в соответствии с *reference collapsing*:

- `T&` для *lvalue*
- `T&&` для *rvalue*

Правило вывода:

- если аргумент *lvalue*, то `T` выводится как `U&`, тип аргумента схлопывается в `U&`
- если аргумент *rvalue*, то `T` выводится как `U`, тип аргумента становится `U&&`
- массивы/функции **НЕ** деградируют до указателя, потому что тип аргумента ссылка

```
int val = 18;
int& ref = val;
const int cval = val;
const int& cref = val;
const int&& crref = 18;

func(val);        // T -> int&,        T&& -> int&
func(ref);        // T -> int&,        T&& -> int&
func(cval);       // T -> const int&,  T&& -> const int&
func(cref);       // T -> const int&,  T&& -> const int&
func(crref);      // T -> const int&,  T&& -> const int&
func(18);         // T -> int,         T&& -> int&&

int* ptr = &val;
const int* ptr_c = &val;
const int* const cptr_c = &val;
int arr[5];
void f();

func(ptr);        // T -> int*&,              T&& -> int*&
func(ptr_c);      // T -> const int*&,        T&& -> const int*&
func(cptr_c);     // T -> const int* const&,  T&& -> const int* const&
func(arr);        // T -> int(&)[5],          T&& -> int(&)[5]
func(f);          // T -> void(&)(),          T&& -> void(&)()
```

Важно отметить, что если `T&&` уточнен квалификатором, например `const T&&`, то контекст вывода типов теряется.

Идеальная передача (prefect forwarding)

Идеальная (прямая) передача (perfect forwarding) - это механизм, позволяющий шаблонной функции-обертке передавать аргументы другой функции, сохраняя их категорию (*lvalue/rvalue*) и квалификаторы (*const/volatile*)

Рассмотрим следующую обертку над функцией

```
template<typename T>
void Wrapper(T&& param) {
```

```
    Process(param);  // param is lvalue = process(int&)
}
```

Даже когда прием происходит по универсальной ссылке, далее при вызове функции `Process` от принятой переменной будет вызов как от *lvalue*, даже если вызов `Wrapper` был от *rvalue*

Для передачи нужного аргумента есть специальная функция `std::forward`

```
template<typename T>
void Wrapper(T&& param) {
    Process(std::forward<T>(param));
}
```

```
template<typename T>
T&& forward(std::remove_reference_t<T>& arg) noexcept {
    return static_cast<T&&>(arg);
}

template<typename T>
T&& forward(std::remove_reference_t<T>&& arg) noexcept {
    static_assert(!std::is_lvalue_reference<T>::value, "bad forward");
    return static_cast<T&&>(arg);
}
```

Отличие от `std::move` в том, что std::move всегда выдает xvalue

```
template<typename T>
std::remove_reference_t<T>&& move(T&& arg) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(arg);
}
```

Что из себя представляет `remove_reference_t` - это псевдоним для типа

```
template<typename T>
using remove_reference_t = typename remove_reference<T>::type;

template<typename T> struct remove_reference { using type = T; };
template<typename T> struct remove_reference<T&> { using type = T; };
template<typename T> struct remove_reference<T&&> { using type = T; };
```

Вывод типов с использованием `auto`

Вывод типа с использованием ключевого слова `auto` работает аналогично выводу типа шаблонов:

- если `auto` не уточнен ссылкой или cv-квалификатором, то ссылки и квалификатор срезаются.
- если `auto&`, то ссылка квалификатор не срезается
- если `auto&&`, то работает как forward reference

Важной особенностью для `auto`, что при инициализации посредством `{}` через `=` будет выведен `std::initializer_list<T>`

```
auto x = {5}; // std::initializer_list<int>
```

В C++14 разрешено указывать `auto` в качестве возвращаемого значения, вывод типа работает аналогично выводу типов в шаблонах

Вывод типов с помощью `decltype`

Ключевое слово `decltype` необходимо для получения типа от имени переменной или выражения, в отличие от `auto` почти не режет типы.

Необходимо для получения типа во время компиляции. Можно поставлять там, где ожидается тип

- decltype(name) - тип с которым объявлено имя переменной
- decltype(lvalue) - тип выражения + левая ссылка `&`
- decltype(xlvalue) - тип выражения + правая ссылка `&&`
- decltype(prvalue) - тип выражения

```
int a[10];
decltype(a[0]) b = a[0]; // -> int& b = a[0];

int x = 18;
// x is name, (x) is expression
decltype(x) c = x; // -> int c = x;
decltype((x)) d = x; // -> int& d = x;

decltype(std::move(x)) e = 1; // xvalue expr -> int&&
decltype(x + 0) f; // prvalue expr -> int
```

```
struct Point { int x, y; };
Point porig {1, 2};
const Point& p = porig;
decltype(p.x) x = 0; // int x or const int& x ? -> int
decltype((p.x)) y = 0; // -> const int& y =0;
```

Чтобы узнать, какой тип выводит `decltype`, можно написать простую шаблонную структуру, приводящую к ошибке компиляции:

```
template <typename T>
struct Error {
    Error() = delete
};

int value = 18;
int& ref = value;

Error<decltype(++ref)> e; // compilation error shows type of decltype(++ref)
```

В C++14 можно использовать `decltype(auto)` для автоматического вывода типа, по правилам `decltype`, а **НЕ** по правилам `auto`

```
double x = 1.0;
decltype(x) tmp = x;
decltype(auto) tmp = x; // double tmp = x;
decltype(auto) tmp = (x); // double& tmp = x;
```

Variadic Template (C++11)

Variadic templates — это одно из самых мощных нововведений C++11, которое позволяет создавать шаблоны, принимающие произвольное количество аргументов любого типа. Это основа для `std::tuple`, `std::function`, идеальной передачи аргументов (*perfect forwarding*) и многого другого.

Пакет параметров (parameter pack) обозначается `...`

```
template <typename... Args>  // Args is template parameter pack
void func(Args... args);   // args is это function parameter pack
```

- `...` означает пакет (pack) - ноль или более аргументов
- `typename...` - принимается произвольное количество типов
- `Args...` - произвольное количество аргументов, но столько же сколько типов

Количество аргументов в пакете можно получить с помощью специальной версии оператора `sizeof...`:

```
template <typename... Args>
void PrintCountArgs(Args... args) {
    std::cout << sizeof...(args) << std::endl;
}

PrintCountArgs();                  // 0 args
PrintCountArgs(1);                 // 1 args
PrintCountArgs(1, 2.5, "hello");   // 3 args with different types
```

Паттерн раскрытия пачки параметров - `args...`

```
template <typename... Args>
void func(Args... args);

template <typename... Args>
void call(Args... args) {
    func(args...);    // -> f(x, y);
    func(&args...);   // -> f(&x, &y);
    func(h(args)...); // -> f(h(x), h(y));
    func(const_cast<const Types*>(&args)...) // -> f(const_cast<const int*>(&x),
...)
}

call(1, 1.0); // -> g(int x, doublee y);
```

Паттерн раскрытия жадно матчит выражение слева:

```
// пусть args - x,y,z
func(h(args...) + h(args)...)
// func(h(x,y,z) + h(x),
//      h(x,y,z) + h(y),
//      h(x,y,z) + h(z))

func(h(args, args...)...)
// func(h(x,x,y,z),
//      h(y,x,y,z),
//      h(z,x,y,z))
```

Рекурсивный вызов

Рекурсивный подход к распаковке параметров заключается в написании базового случая для нуля аргументов и случая когда отщипывается один аргумент

```
void Print() {
    std::cout << std::endl;
}

template<typename T, typename... Rest>
void Print(const T& first, const Rest&... rest) {
    std::cout << first << " ";
    Print(rest...);
}

int main() {
    print(1, 2.5, "hello", 'a');  // 1 2.5 hello a
}
```

Паттерны свертки (Fold expression) C++17

В C++17 введены удобные паттерны свертки использующие `()` и `...`:

- (pack op ...) - (p1 op (p2 op (... (pN-1 op pN)...) - унарная правая свертка
- (... op pack) - (... (p1 op p2) op p3) ... op pN) - унарная левая свертка
- (pack op ... fini) - (p1 op (p2 op (... (pN op fini)...) - бинарная правая
- (init ... op pack) - (... (init op p1) op p2) ... op pN) - бинарная левая

Распечатать все элементы:

```
template<typename... Args>
void Print(Args... args) {
    (std::cout << ... << args) << std::endl;
    // ((std::cout << arg1) << arg2) << arg3 ...
}

int main() {
    Print(1, 2.5, "hello");  // 12.5hello
}
```

Добавить пробелы:

```
template<typename... Args>
void print(Args... args) {
    ((std::cout << args << " "), ...);
    // (std::cout << arg1 << " "), (std::cout << arg2 << " "), ...
}
```

Прозрачная функция

Можно написать прозрачную оболочку:

```
template <typename Func, typename... Args>
decltype(auto) transparent(Func func, Args&&... args) {
    return func(std::forward<Args>(arg)...);
}
```

Нужно когда например нужно вокруг вызова функции добавить замер времени или декораторы.

Можно написать ещё более идеальную функцию, которая принимает функциональный объект

```
template <typename Func, typename... Args>
decltype(auto) transparent(Func&& func, Args&&... args) {
```

```
    return std::forward<Func>(func)(std::forward<Args>(arg)...);
}
```

- Принимает любой вызываемый объект (функцию, лямбду, функтор)
- Принимает любые аргументы для него
- Идеально передаёт их (сохраняет категории значений: lvalue/rvalue)
- Возвращает то, что возвращает вызванная функция с сохранением исходного типа `decltype(auto)`